

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	Total Marks:	100 (1 Question × 100 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Avinash k	Reg. No.:	192411044
Section / Batch:	CSE	Date:	20/7/26

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – Technical Presentation

Q1. Visualization-Based Technical Presentation on Growth Functions

Analyze growth functions $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$ using visualization tools in a technical presentation format. Clearly define the purpose of comparing these functions for understanding algorithm efficiency. Use graphs, tables, or charts to show how each function behaves as the input size increases. Explain which functions remain practical for large-scale problems and justify your reasoning. Present the conclusions clearly and professionally by linking growth behavior to algorithm selection.

Code of Conduct

I Avinash / 192411044 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Avinash

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Marks Evaluation:

Question No.	Technical Content Accuracy (30%)	Problem Identification (25%)	Use of Tools (20%)	Communication (15%)	Professionalism (10%)	Total Marks (100)
Q1	4	4	4	2	3	90
Overall Total (100)						90

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



Technical Presentation

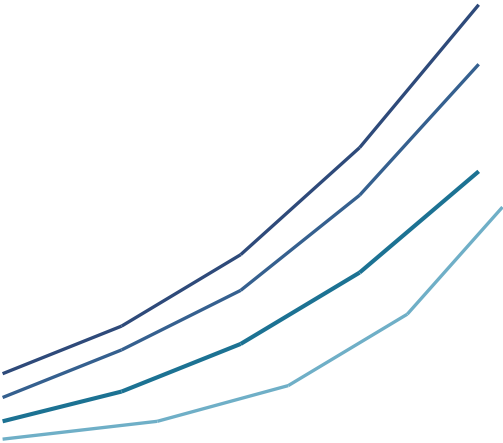


DESIGN & ANALYSIS OF ALGORITHMS

Comparison of Growth Rates

Evaluating $\log n$, n , $n \log n$, n^2 , and 2^n to guide efficient algorithm design

Presented by Avinash (192411044)



PROBLEM STATEMENT

Two algorithms can solve the exact same problem correctly, yet behave completely differently as input size n grows. A function that looks negligible for small n can dominate runtime once n reaches real-world scale — and an algorithm that is “fast enough” in testing can become unusable in production.

- Correctness alone does not guarantee a usable algorithm at scale.
- Constant factors matter less than the shape of the growth curve as $n \rightarrow \infty$.
- Selecting the wrong complexity class can turn milliseconds into years.

GOAL OF THIS ANALYSIS

Compare five growth functions and translate their curves into practical algorithm choices.

$\log n$

Logarithmic

n

Linear

$n \log n$

Linearithmic

n^2

Quadratic

2^n

Exponential

ALGORITHM OVERVIEW

Approach to comparing growth functions

A repeatable procedure to move from raw formulas to a ranked, evidence-based conclusion.

1

Define the functions

State the five candidate growth functions: $\log n$, n , $n \log n$, n^2 , 2^n .

2

Generate a value table

Evaluate every function across an increasing sequence of n (1, 2, 4, 8, 16, 32 ...).

3

Plot the trends

Visualize all curves together to see divergence as n grows.

4

Apply the limit test

Use $\lim_{n \rightarrow \infty} f(n)/g(n)$ to formally rank two functions' growth.

5

Classify & conclude

Order the functions into Big-O complexity classes and map to real algorithms.

Growth comparison procedure

```

FUNCTION compareGrowthRates(N_MAX):
  n_values ← [1, 2, 4, 8, 16, 32, ..., N_MAX]
  FOR each n IN n_values:
    f_log ← log2(n)
    f_lin ← n
    f_nlog ← n * log2(n)
    f_quad ← n * n
    f_exp ← 2^n
    STORE (n, f_log, f_lin, f_nlog, f_quad, f_exp)
  END FOR

  PLOT n vs each f_* on a shared axis
  FOR each pair (f_a, f_b) in candidates:
    ratio ← LIMIT (f_a(n) / f_b(n)) as n → ∞
    IF ratio → 0: f_a grows slower than f_b
    ELSE IF ratio → ∞: f_a grows faster than f_b

  RETURN order ← [log n, n, n log n, n², 2^n] // slowest → fastest

```

ROUTINE COST

Table generation:

$O(k)$ — one pass per sampled n ; k = number of samples

2nd evaluation:

Dominates cost for large n — values overflow standard integer types quickly

Note:

This pseudocode analyzes growth — it is not itself the algorithm being benchmarked. Real target algorithms are mapped in the Applications slide.

WORKING EXAMPLE — DRY RUN

Tracing the values as n doubles

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
1	0	1	0	1	2
2	1	2	2	4	4
4	2	4	8	16	16
8	3	8	24	64	256
16	4	16	64	256	65,536
32	5	32	160	1,024	4,294,967,296

READING THE TRACE

From $n = 1$ to $n = 32$ (a 32× increase in input):

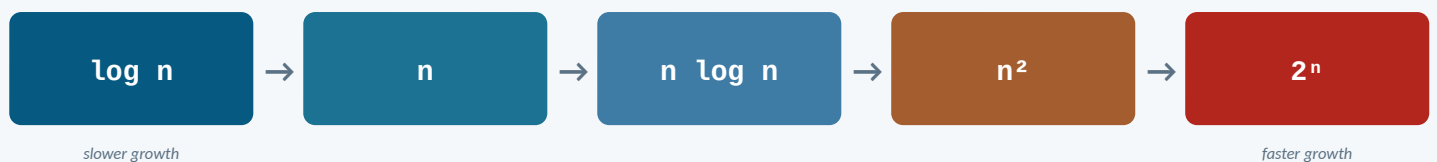
- $\log n$ grows just 5× ($0 \rightarrow 5$)
- n grows exactly 32×, by definition
- n^2 grows 1,024×
- 2^n grows over 2 billion×

Same input growth, wildly different output growth — this is the entire reason complexity class matters more than hardware speed.

COMPLEXITY ANALYSIS

Formal ordering of the five classes

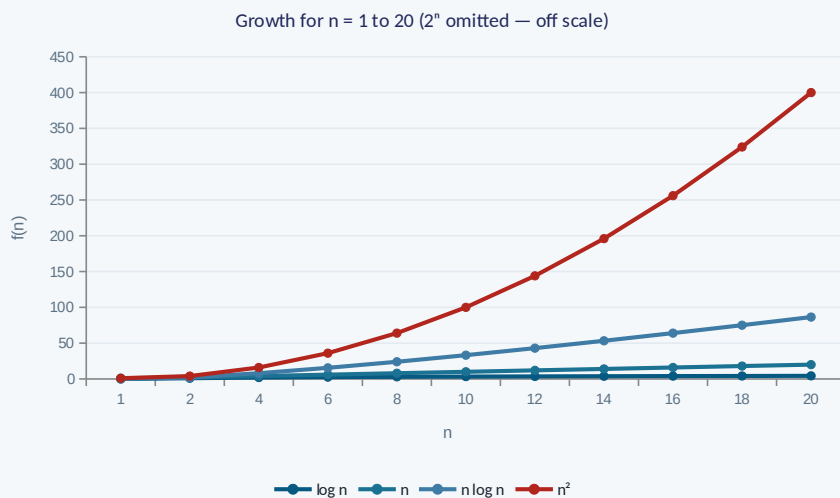
Using the limit test $\lim_{n \rightarrow \infty} f(n)/g(n)$, each function is provably slower-growing than the one to its right:



$O(\log n)$	Input halves each step (e.g. repeatedly discarding half the data).	$O(n^2)$	Nested loop over the input against itself.
$O(n)$	Every element touched exactly once.	$O(2^n)$	Every subset of n items is explored — doubles with each new element.
$O(n \log n)$	Input is split ($\log n$ levels) and each level does $O(n)$ work.		

COMPARATIVE ANALYSIS

Curves diverge fast — and 2^n leaves the chart



2^n — TIME TO COMPUTE

At 1 billion ops/sec:

$n = 10$	~1 microsecond
$n = 20$	~1 millisecond
$n = 30$	~1 second
$n = 40$	~18 minutes
$n = 50$	~13 days
$n = 60$	~36 years

Beyond $n \approx 60$, brute-force 2^n becomes intractable on any classical hardware.

APPLICATIONS

Where each growth class shows up in practice

$\log n$	Binary Search	Locating a value in sorted data
n	Linear Search	Scanning unsorted data once
$n \log n$	Merge Sort / Quick Sort (avg)	General-purpose efficient sorting
n^2	Bubble Sort / Insertion Sort	Simple sorting on small datasets
2^n	Subset Sum / TSP (brute force)	Exhaustive search over all combinations

RESULTS & DISCUSSION

What the comparison tells us

5×

log n growth
from $n=1 \rightarrow 32$

1,024×

n^2 growth
from $n=1 \rightarrow 32$

2.1B×

2^n growth
from $n=1 \rightarrow 32$

- **Logarithmic and linear functions scale gracefully.**

Both remain fast even as n reaches millions — ideal for latency-sensitive operations.

- **$n \log n$ is the practical ceiling for large-scale sorting.**

It balances thorough comparison work with near-linear growth, which is why it dominates production sort algorithms.

- **Quadratic algorithms are fine only for small, bounded n .**

n^2 is acceptable for small datasets but becomes a bottleneck once n exceeds roughly 10^4 – 10^5 .

- **Exponential growth makes brute force unusable at scale.**

2^n is only viable for very small n (roughly ≤ 25 – 30); beyond that, approximation or pruning (e.g. DP, branch-and-bound) is required.

CONCLUSION & REFERENCES

Choosing the right growth class wins at scale

KEY TAKEAWAYS

- Growth rate — not raw hardware speed — determines whether an algorithm scales.
- The ranking $O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$ holds for all sufficiently large n .
- Prefer $O(\log n)$ / $O(n)$ / $O(n \log n)$ algorithms whenever the problem allows it.
- Reserve $O(n^2)$ for small, bounded inputs and $O(2^n)$ for tiny search spaces only.
- When brute force is exponential, look for divide-and-conquer, greedy, or dynamic-programming reformulations.

REFERENCES

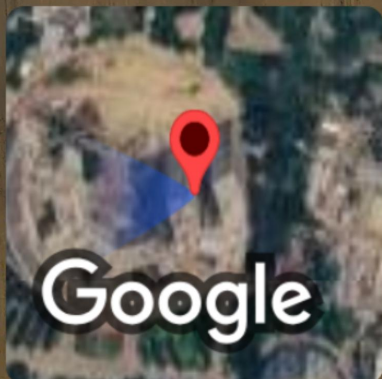
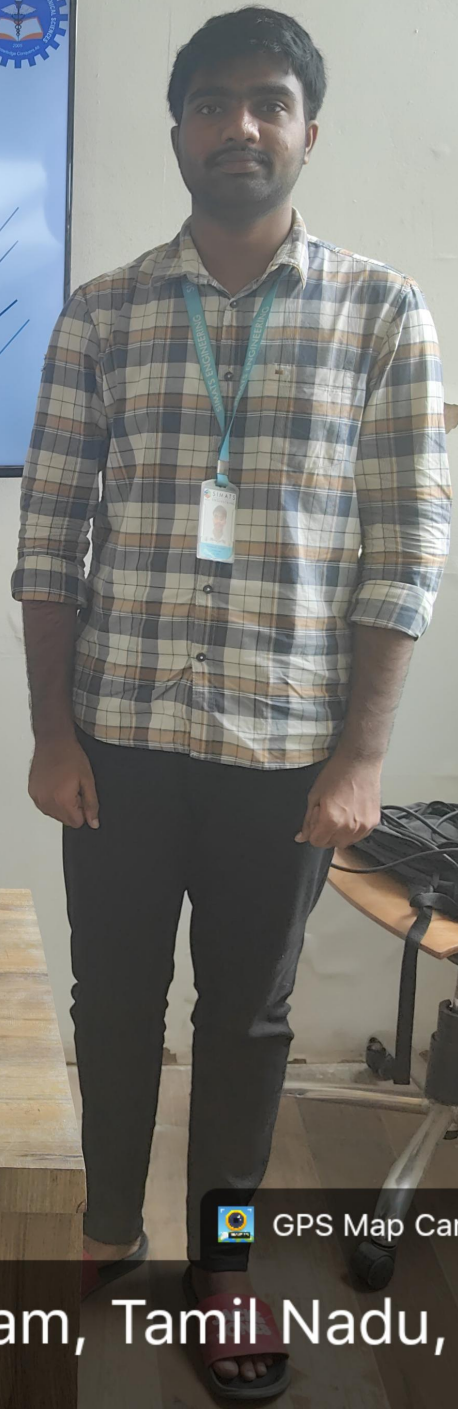
Cormen, Leiserson, Rivest & Stein — Introduction to Algorithms (CLRS), 4th ed., MIT Press.

Sedgewick & Wayne — Algorithms, 4th ed., Addison-Wesley.

Sartaj Sahni — Data Structures, Algorithms, and Applications.

GeeksforGeeks — Analysis of Algorithms (Asymptotic Notation).

Khan Academy — Asymptotic Notation, Computer Science course.



GPS Map Camera

Mevalurkuppam, Tamil Nadu,
India 🇮🇳

22g8+cwh, Mevalurkuppam, Tamil Nadu 602105,
India

Lat 13.026436° Long 80.017°

Monday, 20/07/2026 11:33 AM GMT +05:30